Building a Scalable Recommendation System Using Amazon Reviews Data

Torin Lindsay, Pangkaew Chansri, Emily Gomer

January 2026

Gitlab link: `https://scc-source.lancs.ac.uk/gomere/scc454`

Executive Summary

This project simulates a modern e-commerce platform using the Amazon Reviews 2023 dataset, with a focus on the category All_Beauty. The project is composed of four main tasks along with data preprocessing and vectorisation. The data cleaning pipeline employs dimensionality reduction, text standardisation, and time conversions to prepare the dataset. Various text-to-vector conversion methods were assessed using MRR, identifying TFIDF as the highest-scoring technique.

Task 1 compares three databases (PostgreSQL, MongoDB, and Cassandra) by evaluating their query execution times, showing that MongoDB achieved the lowest overall query execution time. Task 2 implements three separate vector databases (Chroma, LanceDB, and Qdrant) with Qdrant demonstrating the fastest query performance. A similarity search for both user-user and product-product similarity is also produced. Task 3 applies multiple clustering algorithms (BFR, DBSCAN, Bi-secting, and K-Means), and results in distinct product and customer profiles that can be used in business decisions. Finally, Task 4 compares between multiple recommendation systems (a popularity baseline, collaborative filtering (ALS), content-based, and a hybrid model), finding the RMSE and precision at k to determine the best system.

1 Introduction

Online retail systems must be capable of storing and processing hundreds of millions of user interactions and support a diverse range of queries.

This project uses the Amazon Reviews 2023 dataset to simulate a modern e-commerce platform that operates on a large scale. Multiple databases were compared to see how well each of them handle specific queries, how scalable they are, and the complexity of the databases. This was done using similarity comparisons, clustering analysis, and finally making a recommendation system.

Big data is defined by the Big 5 V's: volume, velocity, variety, veracity and value [1]. Challenges occur at many stages due to the size of big data, namely storage constraints, processing, and scalability. The Amazon Reviews 2023 dataset suffers from these challenges with its size of 571 million reviews across 33 categories [2].

Historically, database systems were structured and required physical storage; users had to understand how the data was physically stored to access it. Relational databases implement Edgar F. Codds principles, which made data querying more user friendly [3]. Codd's rules define the core principles of relational databases given ACID (Atomicity, Consistency, Isolation, Durability). This is important for large datasets to prevent inconsistencies and unpredictable behaviour [3].

Relational database management systems are primarily vertical, where an increase in performance or capacity requires additional CPU, memory, or storage. Data is stored on the disk, then put onto the RAM as required during processing. This can be easy to manage as there is only one server. However, there is a single point of failure which can result in data loss if the system fails. To improve relational

database management systems, more CPU power, RAM, or faster disks are required which can be costly to achieve [3] [4] [5].

Modern database systems use horizontal scalability; the data and workload are split across multiple machines. The additional 'nodes' allow for the data to be spread using the replication factor (how many copies of each row there are). This reduces the risk of data loss from a single point failure that may happen in vertical databases [6] [7].

2 Data Description

When finding the characteristics of the data, a smaller subset, all_beauty, was taken to view its schema.

Table 1: Product and User Review Schemas

Product Feature	Type
main_category	str
title	str
average_rating	float64
rating_number	int64
features	object
description	object
price	float64
images	object
videos	object
store	str
categories	object
details	object
parent_asin	str
bought_together	float64

User Review Feature	Type
rating	int64
title	str
text	str
images	object
asin	str
parent_asin	str
user_id	str
timestamp	datetime64[ms]
helpful_vote	int64
verified_purchase	bool

In the meta data, to conserve power, deleting unnecessary columns has been done at the first step. 'Bought_together' (due to missing values), 'images' and 'videos' columns are dropped as they are not needed for analysis.

Handling the 'details' column was challenging due to its nested array structure. Flatten array has been performed which removed columns with over 80% missing data.

The data cleaning pipeline consists of:

- Convert text to string and remove HTML and links
- Convert emoji to their meaning
- Normalising to lowercase
- Tokenisation and removing stop words
- Lemmatisation and elongated words

There is a limitation in the elongated word stage; 'Sooo gooood' turns to 'Soo good' not 'So good' because the criteria are replacing more than 2 repeated letters to make sure 'good' will not be converted to 'god'. Although it cannot guarantee the outcome will be actual word, it reduces the number of identity cases which is useful in vectorisation.

Finally, Review data cleaning. The columns 'images' and 'asin' have been removed because they are not used. The same cleaning pipeline as meta data has been performed. For the timestamp data, it has been converted from milliseconds to seconds and transformed into a standard timestamp datatype. Additionally, the missing values in helpful_vote are replaced with 0. Finally, rating_num and verified_purchase are converted into datatypes double and Boolean respectively.

When choosing a vectorisation method, comparisons were made between four different methods; TFIDF, Word2Vec, Hashing and Principal Component Analysis (PCA), and all-MiniLM-L6-v2. The text values were combined into a single row, the title and the description, then converted into a vector.

3 Methodology

The whole dataset of 571 million data points could not be run, so a smaller subset was taken, one category called All_Beauty.

3.1 Task 1

When choosing and comparing databases to use, a couple core types were looked at: a relational database PostgreSQL [4] [5], a document database MongoDB [8] [9], and a wide-column distributed database Apache Cassandra [6] [7].

MongoDB uses JSON storage, a flexible schema, and has horizontal sharding between machines [8] [9]. It is good at processing semi-structured JSON reviews as it allows iterations on the schema as the transformation of data during ingestion is minimal. However, the enforcement of constraints is limited, and the aggregation is often slower than SQL-based databases that are optimized for complex, multi-table joins.

PostgreSQL [4] [5] has strong ACID compliance, a structured schema, and robust foreign key relationships. It is good at complex aggregation queries, keeping the integrity of the data, and optimisations. However, it fails at horizontal scaling, complex joins may fail at scale, and it runs the risk of single point failure which could lead to the loss of access to the dataset unless you back up the data with other third-party tools, increasing the complexity of the final product.

Like MongoDB, Apache Cassandra uses horizontal scaling, has partition tolerance, and operates under masterless architecture and partitions data using hashing – the partition key [6] [7]. It is best at linear scalability, fault tolerance from replication, and write throughput. However, it doesn't use joins, a query driven schema is required where you must model your data specifically to satisfy the query patterns you already know you will need, and the aggregation capability is limited.

3.2 Task 2

The main objective of Task 2 was to create both a user-user similarity search system, and a product-product similarity search system.

When deciding which vector databases to use, there were issues surrounding finding free databases. A decision was made to implement Chroma, LanceDB, and Qdrant. Many of the other databases, such as Weaviate and Milvus, required a paid subscription for their managed cloud-based systems and did not offer a quality and differentiable local-based solution. However, if the scalability of the data is in question and there is a lack of CPU power available, a paid cloud-based version may be more appropriate for a final product on accessing all the data.

One main technical issue was that many of the databases chosen had their own prebuilt vectorisation methods automatically loaded into their systems or had no default vectorisation method; Chroma had all-MiniLM-L6-v2, LanceDB and Qdrant had none by design. This issue was resolved by instead creating our own models, since it was not valid to use the inbuilt functions for this project. To diversify, a comparison was made between TFIDF to a sentence transformer method (all-MiniLM-L6-v2), an approximating method (Hashing and Principal Component Analysis (PCA)), and an embedding method (Word2Vec). Through a comparison of these methods, TFIDF was the chosen vectorisation method.

For scalability, an Approximate Nearest Neighbours method of Hierarchical Navigable Small World (HNSW) was implemented. This was standardised across all the databases for a fairer comparison, except for LanceDB which first partitioned the data into 128 sections before doing HNSW due to its unique structure. 'ef_constuction' was set as 100 to balance between the recall values and the memory constraints. Increasing this value would increase the recall of the search but would make the data ingestion slower and more CPU intensive.

Chroma was used as a database for its easy-to-use querying and ingesting functions. Firstly, a client space on the local disk was created. A collection on this client space was created that used HNSW. The

pre-calculated TFIDF vectors were added into this collection at a batch rate of 5000 since the limit for the size of the batches was capped at 5461 by Chroma. A final query was made on the results to find which users or products were the most similar between the target user or product.

LanceDB was selected for its unique storage architecture. Unlike traditional row-based databases, LanceDB utilises the Lance columnar storage format, which is specifically optimised for high-performance workloads. This columnar design minimises input and output overhead by indexing, which significantly accelerates similarity comparisons. By setting an index on the vectors that have been created, it allows the database to only parse through those vectors to find the desired vector, and further comparisons for similarity can be done quicker.

Unlike LanceDB and Chroma, Qdrant has a free-to-use cloud server to store your data indefinitely. Since the data was being stored on a cloud server, a consideration about security had to be made with regards to the API key leaking. This was resolved by creating a .env file that stored this key. Qdrant required numerical IDs so a hashing algorithm, SHA-256, was implemented. By using utf-8 encoding techniques on long digits such as the ASIN column, it allowed Qdrant to compare and identify different ASIN's. By using these encoded ASIN's, it could match them to the vectors used.

3.3 Task 3

The main objective of this task was to cluster the product and review data to inform business decisions.

3.3.1 Product segmentation

Columns 'average_rating' and 'rating_number' were selected to identify product popularity and quality reliability by using BFR and DBSCAN. Semantic analysis has been performed by applying Multi-Stage K-Means and BFR to the product title and features. This step can be done by removing customised stop words such as 'oz' and 'ml', which is necessary because some frequent words are meaningless, so removing them allows the model to perform better. The reason that K-Means is not selected in this stage is because after the first clustering, the best K value is two, which is simpler for large scale data as more than 80% of total data belong into one cluster. Therefore, multi-stage K-Means has been performed by re-clustering the data from the first round, aiming to get more nuanced clusters.

3.3.2 Review segmentation

K-means and Bi-secting K-mean are performed with the review data for a comparison between agglomerative and divisive algorithms. To perform review clustering, new features have been created: 'seasonal shopper', 'customer recency', 'semantic score', and 'text length'. By extracting the month component from the 'timestamp' feature and mapping month into each season, it is stored in the 'seasonal shopper' column. The difference between current date and review date has been used to create customer recency, identifying customer life cycle. Rating has been converted to a new numerical value, -1, 0 and 1 using the criteria that if rating is greater than 3.5, it will be converted to 1, lower than 3.5 will be –1, and 0 for remaining cases.

To select the correct k-value for K-Means and Bi-secting K-Means a loop over k = 2 to 10, a k-elbow graph, and silhouette score are used to evaluate each k-value. The k-value with the highest silhouette score is the selected hyperparameter with the optimal k-value [10]. Finally, since every clustering execution can provide a different result, randomseed=42 was selected for replication.

3.4 Task 4

The aim of task 4 was to develop, evaluate and compare multiple recommendation systems.

Recommendations can be personalised, where suggestions are tailored to individual users' preferences or non-personalised based on overall popularity and trends. This project implements both, where non-personalised is used as a baseline to investigate how personalisation can improve the relevance and quality of the recommendations.

Exploratory data analysis was carried out to calculate the number of ratings, total number of users, total number of products and the sparsity of the user-item matrix. Most users only rate and use a small number of products, making the matrix sparse.

Recommender systems can learn from either explicit feedback where products are rated and reviewed, or implicit feedback which learns from user actions such as little time on a product page suggests low interest, whereas clicking and purchasing indicates higher interest. This project used explicit and the dataset was split into training, validation, and test sets to enable fair evaluation and hyperparameter tuning.

3.4.1 Random Baseline

A random recommender was implemented as a baseline where for a given user, the system randomly selects products that the user hasn't rated. This system uses no information about the user or product.

3.4.2 Popularity-based baseline

Popularity baseline was developed where each product was ranked according to the average rating and number of reviews, which was combined for a popularity score. This method recommends highly rated products and frequently reviewed products and excludes products that have already been rated or seen by the user.

This works well for new users with no/little interaction history but is not personalised.

3.4.3 Content-based filtering

A personalised content-based filtering approach was implemented using product titles and descriptions to recommend using products features and user preferences.

A profile vector was created for each user by averaging the TFIDF vectors of products they rated highly, and the cosine similarity calculated distance between the user profile and other product vectors in the vector space. This approach recommends similar products to users based on their highly rated products. However, there is no cold start for new items that match existing similar products.

3.4.4 Collaborative filtering with ALS

A personalised collaborative filtering approach was developed, which recommends items based on similarities between users or items. If two users have similar preferences, such as purchasing similar beauty products, the system assumes that items purchased by one user may interest the other.

This was developed using Alternating Least Squares (ALS) which was trained on the user-product rating interactions. ALS factorises the user-item matrix into latent factors which gives personalised user preferences and product features. This system predicts ratings for unseen items to generates personalised recommendations.

Several hyperparameters influence the ALS model performance: rank (the number of latent factors) and regParam (the regularisation parameter). A grid search approach was used for hyperparameter tuning where multiple combinations were tested and evaluated using RMSE on the validation set. The configuration that produced the lowest RMSE was selected, since that indicated a better prediction accuracy.

3.4.5 Hybrid recommender

A hybrid recommender system was implemented to provide personalised and varied recommendations to both existing and new users by combining both approaches. The final recommendation score was calculated using a weighted combination of both models.

A parameter α was used as the weight to balance the two models, where α =1 was a purely content-based recommender and $\alpha = 0$ purely collaborative filtering. Various values of α were tested and evaluated to see how the weights of the models affected the overall performance.

RMSE treats all prediction errors equally and does not directly measure the quality of the recommendations. Therefore, relevance of the recommended products needed to be evaluated using precision at k ranking metric which measures the proportion of the top k recommended items that are relevant. Items rated $\geq$4 were considered relevant when calculating Precision@10.

4 Results and Analysis

4.1 Task 1

To choose the best database for our situation, a timing of how long each database took to run each query was made. It did not time how long it took to ingest the data since in a real-world scenario as the database would already have the data in.

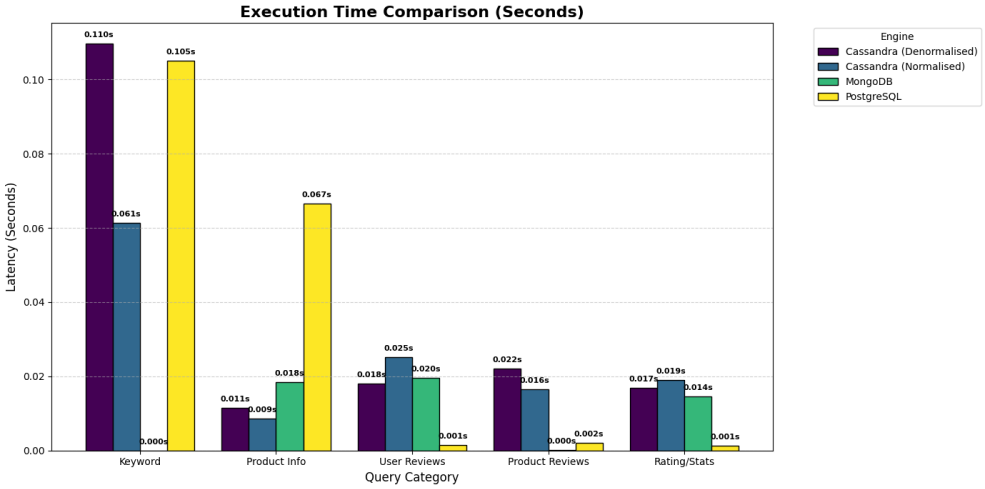

Figure 1: Query Latency Visualization

Table 2: Detailed Query Performance Values

Query	Cassandra (D)	Cassandra (N)	MongoDB	PostgreSQL
Keyword	0.109723568	0.061454058	3.05E-05	0.105151176
Product Info	0.011496067	0.008658409	0.0183589	0.066627979
User Reviews	0.018063784	0.025197744	0.0196331	0.00144887
Product Reviews	0.022074699	0.0164783	7.37E-05	0.001986027
Rating/Stats	0.016877413	0.01901269	0.0144997	0.001348257
Totals	0.178235531	0.130801201	0.0525959	0.176562309

The above graph shows the execution time for each of the five queries; Keyword, Product Information, User Reviews, Product Reviews, and Ratings. For further comparison, two tables are provided below the graph with more exact values.

The first table shows that MongoDB has the lowest value out of all queries; it is significantly less than all the other databases in not only those queries, but also every other query done.

Cassandra has both a Normalised and Denormalised version for a comparison between whether normalisation helps reduce the total query time.

As seen in the second table, MongoDB has the lowest overall timings for all five queries. MongoDB's total time is roughly 2.5 times faster than the next closest competitor and 3.4 times faster than PostgreSQL. For a real-world scenario where query latency directly impacts user experience, this margin is significant.

4.2 Task 2

To choose the best database for our situation, a timing of how long each database took to run each query was made. It did not time how long it took to ingest the data since in a real-world scenario as the database would already have the data in.

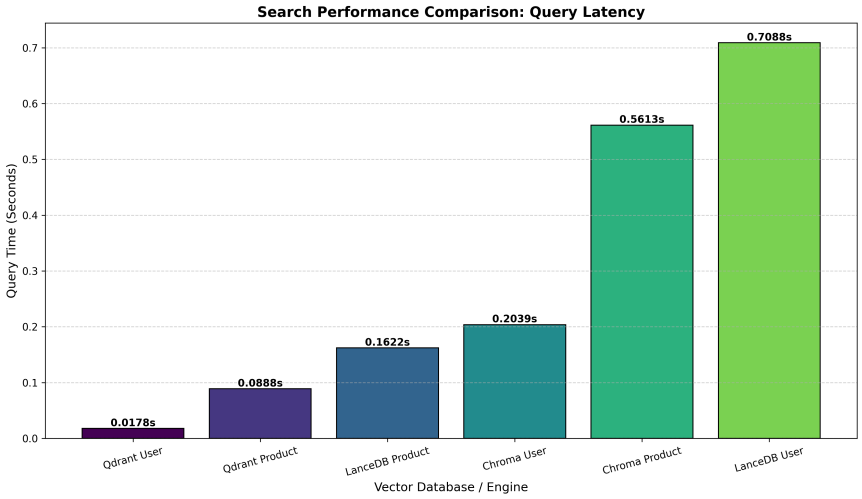

Figure 2: Vector Database Performance: User vs Product Search

Table 3: Detailed Latency (Left) vs. Total Aggregated Time (Right)

Engine	Time (s)
Qdrant User	0.01781559
Qdrant Product	0.08884263
LanceDB Product	0.162170887
Chroma User	0.20390892
Chroma Product	0.561287403
LanceDB User	0.708806276

Engine	Total Time (s)
Qdrant	0.1066582202911377
Chroma	0.7651963233947754
LanceDB	0.8709771633148193

The above graphs show the query timings for both users and products for each database, as well as a joint graph to show the total time for both. Overall, it shows that for both queries in users and products, Qdrant had the least time. LanceDB, however, has a clear separation in user and product queries. This may be due to the scalability of LanceDB not being as good.

Whilst Qdrant had the clear best query timings, it is not representative of the whole truth. Qdrant was run on the cloud for a diverse selection. Theoretically, Chroma and LanceDB should also be better on the cloud, however since they do not have a free version we cannot test if that is true. For the size of the subset chosen, the Qdrant cloud server is perfect to use, however if the whole dataset is needed to be run, then a paid version is required. If a cloud-based version is not available, then the local-based Chroma version may be the best solution.

Table 4: Top 10 Product Results (Left) and User Results (Right) by Similarity

Rank	ASIN	Product Title	Sim
1	B07QWBWKVW	wide tooth comb curly hair nat..	0.927
2	B07BJ492JM	wood comb wooden hair comb 100..	0.921
3	B07BF4T557	wood comb wooden hair comb 100..	0.921
4	B07WPL2LZJ	wood comb anti static natural ..	0.908
5	B07S7P8NJ5	xuanli hair comb detangling wi..	0.901
6	B07PPBJFWC	xuanli hair comb detangling wi..	0.901
7	B079VHXTXP	wood comb wooden hair comb 100..	0.9
8	B07QX9XTKR	natural buffalo horn comb ruby..	0.899
9	B0778RTNWT	thee natural horn comb anti st..	0.892
10	B0778HW8QF	thee natural horn comb anti st..	0.892

Rank	User ID	Sim
1	AHV6QCNBJNSGLATP56JAWJ3C4G2A	0.706
2	AGPVUWYFDS4WHWJJC4336KQUTAPQ	0.678
3	AFS5MAXWHQNKTORY7CNERRHNV7DA	0.671
4	AEDRFBA64VH2SHGAB5W27J4ZTKHA	0.667
5	AFKSNZYMIZ6WAU3A7RREIJ3RIABA	0.665
6	AHY6N2ZTVLFE7N5OWEPCG2C3ODTQ	0.665
7	AHSMHVELSPKUCSUPDPNRJE2JOIFQ	0.665
8	AF7SECCKJFGECV6A7YJEZL2NWFXA	0.663
9	AHYCC2Y7SQ6QRFSUKDTAUWWAH2FA	0.662
10	AGQVY4TAXPWFHDCKVXJG5KMXBKFA	0.662

The similarity search for both users and products are shown in Table 4. It shows the similarity (distance) between the closest vectors.

4.3 Vectorisation

When choosing a vectorisation method, comparisons between four models were made: TFIDF, Word2Vec, Hashing and PCA, and all-MiniLM-L6-v2.

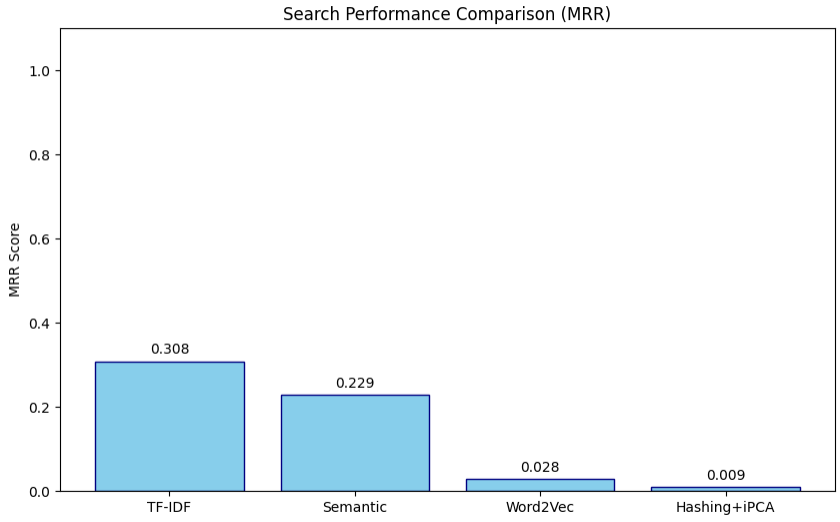

Figure 3: Mean Reciprocal Rank (MRR) Scores across Search Models

Table 5: Detailed MRR Scores for Retrieval Models

Search Model	MRR Score
	0
TF-IDF	0.30833333333333335
Semantic	0.22916666666666666
Word2Vec	0.027777777777777776
Hashing+iPCA	0.009259259259259259

As shown in the graph, TFIDF has the highest Mean Reciprocal Rank (MRR). Whilst TFIDF has the highest value, the value is still quite low. Further improvements through stop words and hashing methods can be used to potentially increase the MRR value.

It was decided that Hashing and PCA was to be compared against due to the sparsity of the resultant vectors. It was by using the PCA that the dimensions of the vector were reduced. This allowed us to project the sparse data into a lower-dimensional, dense representation that captures the most significant patterns while discarding redundant features.

4.4 Task 3

For the product data, there were two approaches: numerical and text.

In the numerical data, avg_rating and rating_number were selected to represent the numerical features. They are classified using BFR and DBSCAN.

Table 6: Clustering Distribution with BFR (Left) and DBSCAN (Right)

Cluster	Reviews	%	Avg
Massive Rising Stars	41824	37.147171	4.042282
Underperforming	33435	29.696243	2.789126
Niche Product	224	0.198952	4.522956
Emerging Risky Product	165	0.146549	2.731996
Premium Rising Stars	36942	32.811084	4.48907

Cluster	Reviews	%	Avg
Core Product	75999	67.50066613	3.890125
Niche Product	604	0.536459721	3.846523
Secondary tier Core Product	35987	31.96287415	3.870092

In term of quality and interpretability, BFR classifies products based on both sentiment and volume, resulting in the five clusters. DBSCAN groups products into three clusters based on volume regardless of average rating. Core product has the largest number of reviews, followed by Secondary tier and Niche Product.

Identifying Risky Product with high engagement allowed for nuanced intervention. Since we know these products have high engagement, the company can prioritise improving product quality over marketing to convert them into Rising Stars.

In the text data, Title, features and description were selected to represent text features. They were classified using BFR and normal k-means.

Table 7: Side-by-Side Comparison of Clustering Metrics with BFR (Left) and K-Means (Right)

Cluster	Mean	Med	M(R)	Sum	Sz
Premium Rising Stars	4.11	4.2	60.37	1973477	32687
Massive Rising Stars	3.97	4.1	58.15	2268733	39016
Underperforming	3.54	3.6	27.68	799294	28877
Moderate Risks	3.82	3.9	48.33	387711	8022
Emerging Risk	3.8	3.9	65.51	261235	3988

Cluster	Mean	Med	M(V)	Sum	Sz
Market staples	3.91	4	42.73	3301534	77263
High engagement risk	3.59	3.6	49.55	107925	2178
Underperforming	3.45	3.5	24.94	150938	6051
Mass-market intermediaries	3.78	3.9	50.6	285419	5641
Premium performance stars	4.1	4.2	87.98	594689	6759
Reliable standard items	4	4.1	60.11	185915	3093
highly-discussed essentials	3.79	3.9	63.27	217191	3433
ViralProduct	3.9	4	105.22	808165	7681
Niche Product	3.95	4	78.77	38674	491

In term of quality and interpretability, BFR algorithm group products with high rating into two groups, while multi-stage K-Means classified them into four groups. More segments allowed the company to identify whether it is viral, niche, market staples, or premium performance for a better business decision.

The company can directly improve products' quality in High-engagement Risks cluster to protect brands because they are being talked about a lot but have low ratings. Additionally, Viral products can be used to advertise to new users, increasing the possibility of becoming a brands' customer. Market staples cluster allows companies to manage inventory by making sure they have enough stock.

For the review data, there were two approaches: seasonal and reviewer type.

In the seasonal profile, K-Means and Bisecting K-Means have been selected to be clustering algorithms to classify selected features, 'average recency' and 'seasonal shopper', into meaningful business insights.

Table 8: Side-by-Side Review Clustering Distribution with K-Means (Left) and Bi-Secting K-Means (Right)

Cluster	Size	Recency	Profile (%)
winter customer	168128	1670 days	89 winter
autumn customer	126889	1683 days	100 Autumn
base line customer	50082	1694 days	Balanced(21-24)
spring customer	142949	1676 days	100 Spring
summer customer	143938	2157 days	100 summer

Cluster	Size	Recency	Profile (%)
Summer customer	143621	1676 days	100 Summer
Winter customer	148763	1708 days	100 Winter
Spring customer	153610	1730 days	100 Spring
Autumn customer	138525	1729 days	100 Autumn
Long-Recency Winter customer	12060	2200 days	100 Winter
Summer customer	35407	1673 days	33 Summer

While K-Means successfully classifies customers based on their behaviour and identifies a baseline group, it struggles with high-variance clusters. Bi-secting K-Means performs better as shown in the table; it distinguishes between long-recency and standard winter shoppers.

The company can use retention strategies to maintain baseline customers or prevent them becoming dormant customers. Additionally, some marketing strategies such as 'win-back campaigns' can be used to encourage Long-Recency winter customers. The benefit of these segments not only provide the company with which customer group should have a 'win-back campaign', but also what type of product should be advertised to them.

For the type of reviewer, K-Means and Bisecting K-means were selected. 'Average Sentiment score' and 'review_length' were chosen to represent features.

Table 9: Side-by-Side Review Clustering Distribution with K-Means (Left) and Bi-Secting K-Means (Right)

Cluster	Size	Sent	Len	Total
Premium Review	122	0.778688525	319.137812	36.42
Standard Review	631864	0.5	98.5580322	1.1

Cluster	Size	Sent	Len	Total
Premium Negative Review	16212	-0.589069825	387.9194776	1.44
Premium Positive Review	31187	1	453.3342939	1.37
Standard Positive Review	414102	1	70.48798408	1.08
Standard Negative Review	170485	-0.702184943	74.4810681	1.1

Both algorithms successfully classified reviews into Premium and Standard Review. While K-Means simply groups reviews into two clusters, Bi-Secting K-Means provides more details by categorising both groups into positive and negative feedback.

The company can focus on Premium Negative Review to understand specific pain points, as these reviews provide more detailed feedback. However, Standard Negative Review likely represent general or high-frequency issues that do not require long text explanations.

4.5 Task 4

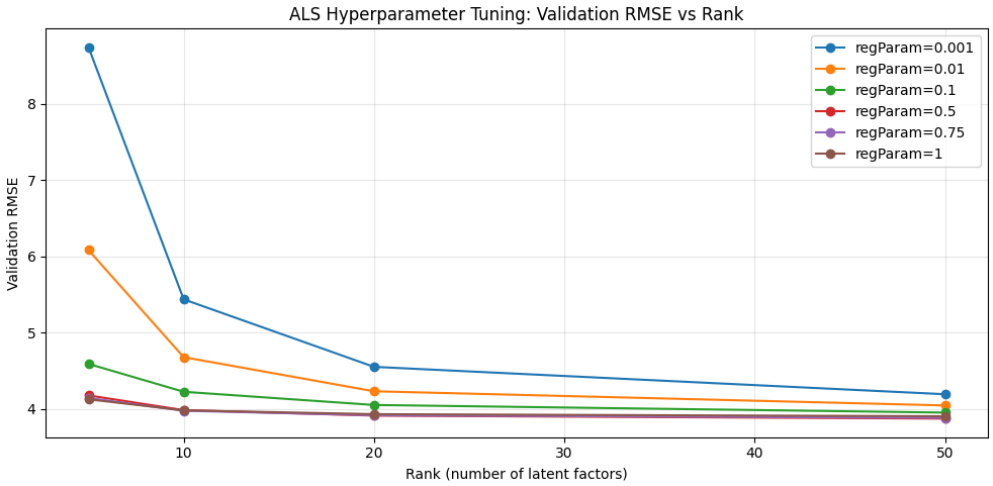

Figure 4: Validation vs Rank: ALS Hyperparameter Tuning

The above graph shows the ALS alpha tuning for different regParams. It shows how increasing the number of latent factors affects the validation RMSE.

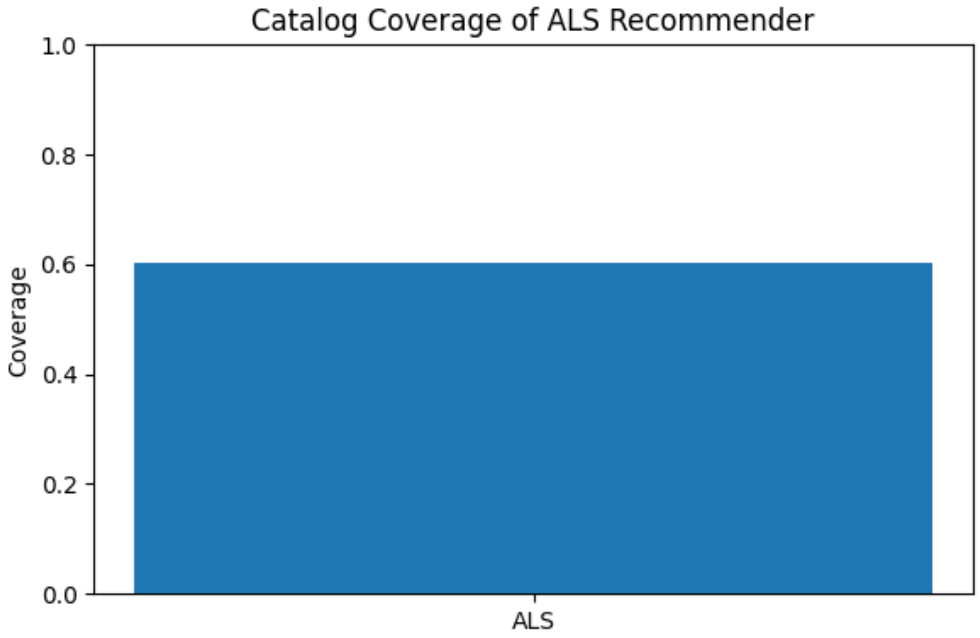

Figure 5: ALS Coverage Graph

The coverage graph above shows what percentage of the items in the dataset can be found by the recommender. This may affect users with niche tastes.

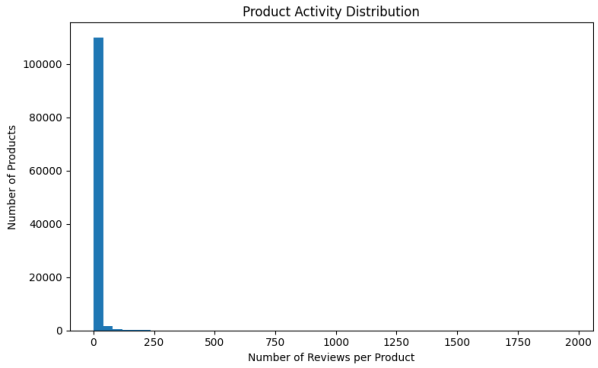

Figure 6: Product Activity Distribution

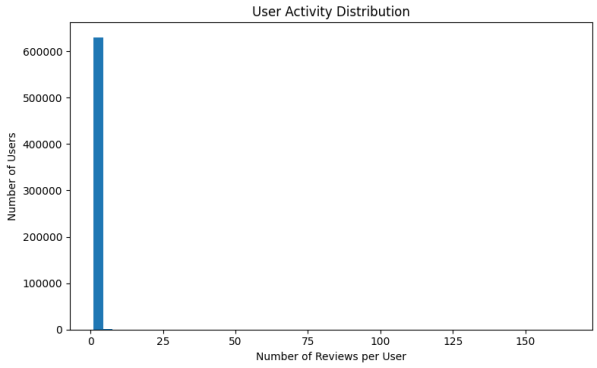

Figure 7: User Activity Distribution

Comparison of Sparsity in Product and User Engagement

The product and user Activity distributions show the number of reviews by product and user and is heavily right skewed, meaning that the data set is extremely sparse.

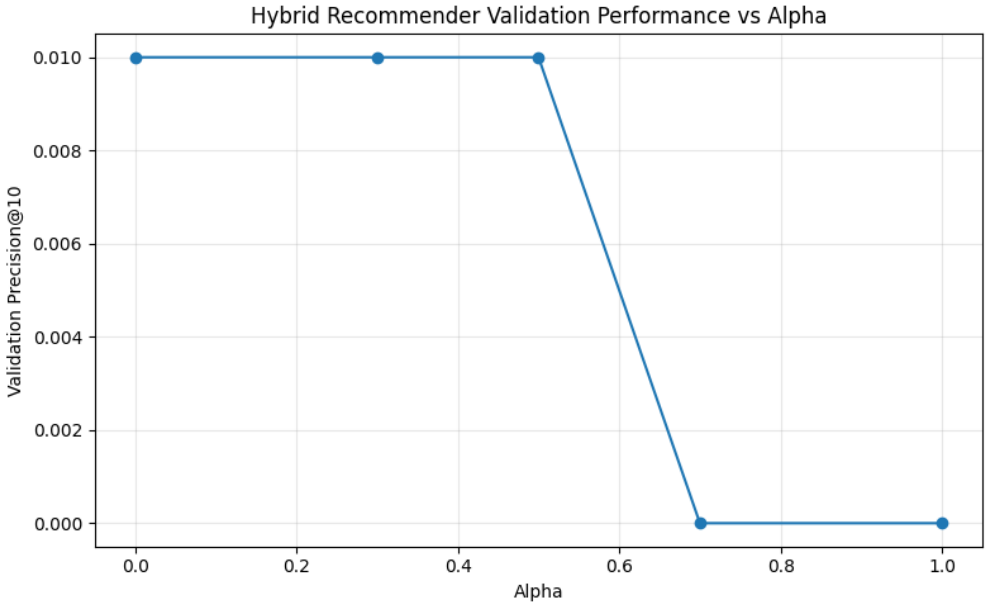

Figure 8: Precision at Certain α values

Figure 8 shows the best α value for the final hybrid recommender. There is a sharp performance drop when α exceeds 0.5.

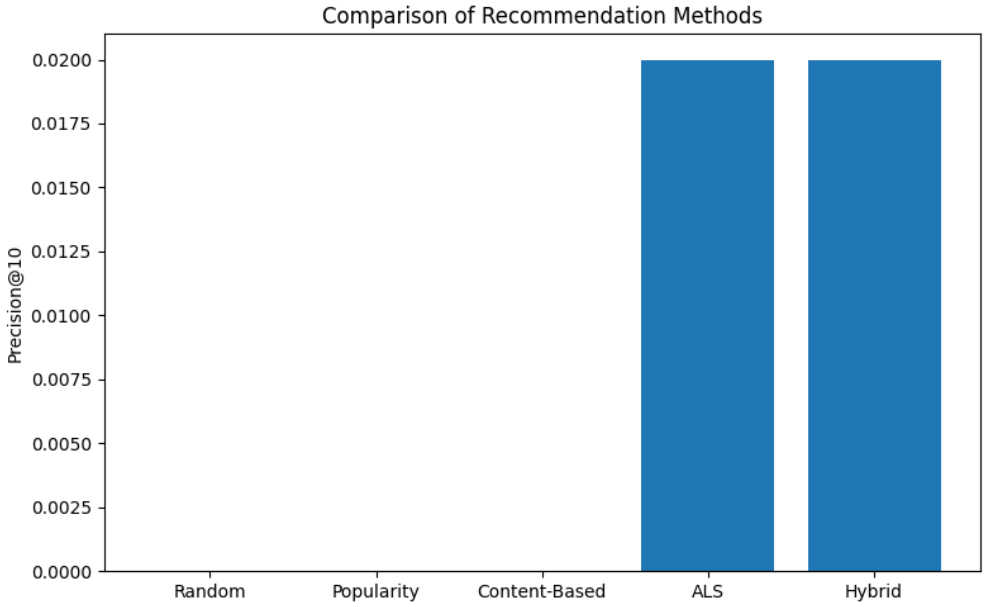

Figure 9: Precision at Certain α values

Figure 9 compares Precision@10 for all recommendation system models. Performance is low due to the sparsity of the data.

5 Discussion

A limitation of this study was the restricted number of database architectures evaluated in Tasks 1 and 2, as well as the manual implementation of vectorization rather than leveraging in-built database features.

The initial evaluation across all-MiniLM-L6-v2, TFIDF, Word2Vec, and Hashing and PCA yielded a consistently low MRR. This suggests that although the models work in mapping the vectors, the ground truth is barely being found.

BFR is suitable for datasets with outliers, as it does not force every data point into each cluster, but it keeps outliers in Retained Set. However, its implementing complexity is high.

DBSCAN is a highly effective clustering algorithm to detect outlier. However, it ignores sentiment features, which means it is not suitable for using when sentiment analysis is required.

Both of K-Means and Bi-secting K-Means are straightforward to implement using standard library. However, they are not suitable for scalability as computational costs increase significantly with data size. Additionally, as shown in Table 8, K-Means fails to handle data with high-variance effectively. In contrast, Bisecting K-Means performs better in this scenario as it can classify two k-means clusters into another 2 sub-clusters.

The Collaborative approach is challenged by cold-start problems when new users or products have no history of interactions or behaviours to compare against similar users.

The popularity-based model can be used to recommend trending and generally popular products to new users who have no interaction history. For new products with similarities to existing items, the content-based approach can recommend items based on similarities between product descriptions and user product preferences. However, these methods rely on good feature vectors that represent the profiles of the items and users well, and can't use other users' judgements. These approaches can also lack diversity, and recommendations may become repetitive and lead to the user to stop using the application.

6 Conclusion

This project successfully simulated a scalable e-commerce data pipeline, revealing distinct performance advantages among various database technologies and machine learning approaches. MongoDB emerged as the most efficient database for executing general e-commerce queries, while Qdrant provided the fastest performance for vector-based similarity searches. The application of clustering algorithms like Bi-secting K-Means and the development of a Hybrid collaborative filtering system successfully transformed raw data into usable customer segments and highly personalised product recommendations.

The personalised ALS outperformed all the models, but the performance was low across all models. The hybrid model did not outperform ALS, as the optimal weighting parameter ($\alpha = 0.0$) reduced the hybrid approach to pure collaborative filtering. However, the low performance is due to the extreme sparsity of the interaction matrix which was revealed in the exploratory data analysis with approximately 99.999% sparsity.

For both Task 1 and Task 2, more databases could have been added in for the comparison. For Task 1, a graph database such as Neo4j, a Full-Text Search Database such as Elasticsearch, or a Key-value store database such as Redis would be good to further compare against. For Task 2, expanding the number of databases to include a Pure Similarity Search Library such as FAISS, a Fully Managed Proprietary Vector Database such as Pinecone, or a Hybrid Search Engine such as Elasticsearch may result in a better database being chosen. Furthermore, the vector databases have their own in-built vectorisation and ANN. Using these could reduce the complexity of the code, as well as reduce the overall run-time.

More vectorisation methods such as BERT or OpenAI embeddings can be used. Optimisation methods can be used to increase all the MRR scores, such as making a list of stop words or expanding the testing set. Finally, a joint hybrid vector method may have been better than the singular methods. For example, doing TFIDF to reduce the dataset down to the top 10000, then using Word2Vec to reduce this further. A result can be found using a Cross-Encoder.

The methods in this project demonstrate the fundamental principles of recommender systems. However, they are traditional approaches and more recent research studies the deep learning and large language model (LLM) based recommendation systems, which capture more complex relationships between users and items [10]. Although deep neural networks enhanced personalisation, the development of large language models have revolutionised natural language processing and artificial intelligence. Neural embeddings and transformers can represent users and products in vector space to gain a semantic understanding of user preferences to tailor recommendations with more accuracy and relevance [11].

References

[1] T. Nguyen and S. Mohamed, "A framework for five big v's of big data and organizational culture in firms," in *2018 IEEE International Conference on Big Data (Big Data)*, pp. 2367–2374, 2018.

[2] Y. Hou, J. Li, Z. Liu, J. Tang, W. X. Zhao, J.-R. Wen, and J. McAuley, "Amazon reviews 2023: A large-scale real-world dataset for product recommendation and beyond," *arXiv preprint arXiv:2403.03952*, 2024.

[3] S. Yu, "Acid properties in distributed databases," seminar paper, University of Helsinki, Department of Computer Science, 2009. Advanced eBusiness Transactions for B2B-Collaborations.

[4] PostgreSQL Global Development Group, "Postgresql: The world's most advanced open source database," 2026.

[5] R. O. Obe and L. S. Hsu, *PostgreSQL: Up and Running: A Practical Guide to the Advanced Open Source Database*. O'Reilly Media, 3rd ed., 2017.

[6] Apache Software Foundation, "Getting started | Apache Cassandra documentation," 2026.

[7] S. Matthews and N. Nakrani, *Learning Apache Cassandra: A Hands-On Guide to High-Level Architecture, Just-In-Time Data Modeling, and Sharding*. Packt Publishing, 2015.

[8] MongoDB Inc., "Mongodb atlas: Cloud document database," 2026.

[9] K. Deepalakshmi and G. Suseendran, "A review on various aspects of mongodb databases," *International Journal of Engineering Research & Technology (IJERT)*, vol. 8, no. 5, pp. 25–29, 2019.

[10] F. Isinkaye, Y. Folajimi, and B. Ojokoh, "A comprehensive review of recommender systems: Transitioning from theory to practice," *Information Advising and Systems*, vol. 3, no. 4, pp. 261–271, 2015.

[11] L. Wu, Z. Zheng, Z. Qiu, H. Wang, H. Gu, T. He, F. Zhu, C. Chen, Z. Zhao, J. Yi, E. Huang, Q. Xu, W. Shen, X. He, Y. Sun, C. Luo, B. Zheng, E. Chen, and H. Zhao, "Recommender systems in the era of large language models (LLMs)," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 9, pp. 4453–4473, 2023.